

AGENCYOS

Orchestrator / Router Agent — Complete A-to-Z Responsibility Specification

Phase 1 Workforce Control Layer | Large → Small → Smallest

Mission: Control how AI work is routed and executed across AgencyOS. Understand the task, match the required capability, select the eligible specialist, select an eligible model/provider, validate permissions and policy constraints, construct the execution envelope, execute safely, validate the result, retry/fallback when permitted, and return a traceable outcome.

Source boundary: Based on the uploaded AgencyOS AI Agent Architecture, Orchestrator/Multi-Model Routing, CRM, Workflow, Security, Automation, Approval and QA specifications. The source architecture makes routing/provider/model selection an Orchestrator responsibility; specialist agents retain domain execution authority; Coordination retains workflow handoff/dependency responsibility.

1. What the Orchestrator Is

- AI workforce routing and execution-control layer.
- Understands task intent and required capability.
- Matches tasks to eligible agents.
- Chooses model/provider according to configured routing rules.
- Builds/validates structured execution envelopes.
- Controls execution, timeout, retry and fallback.
- Validates structured results and completion evidence.
- Maintains routing/execution traceability.
- Optimizes configured quality, reliability, latency and cost trade-offs.
- Escalates when no safe/authorized route exists.

2. What It Is NOT

- Not the client-facing Sales Agent.
- Not the Scheduler Agent.
- Not the Quotation Master Agent.
- Not the Coordination Agent.
- Not the Policy Engine or Admin approver.
- Not a replacement for Finance or later Project Manager workflows.
- Cannot invent requirements, prices, discounts, payment facts or client acceptance.
- Cannot bypass authorization because a model recommends an action.

3. Orchestrator vs Coordination Agent

Question	Orchestrator / Router	Coordination
Who/how executes AI work?	Primary	No
Which specialist?	Selects	No
Which model/provider?	Selects	No
Workflow next step?	Supports execution	Primary
Agent handoff/dependencies?	Execution-level	Primary workflow layer
Retry/fallback route?	Primary	Tracks outcome
Domain business work?	No	No
Policy override?	No	No
Audit	Routing/execution	Workflow/handoff

4. Task Intake

- Accept a valid task/event/execution envelope.

- Identify tenant/organization.
- Identify lead/client/project scope when applicable.
- Identify source actor/agent/system.
- Identify task type.
- Identify intent.
- Identify workflow state.
- Identify correlation ID.
- Identify required output schema.
- Identify priority/deadline.
- Identify context/artifact references.
- Identify policy/permission constraints.
- Identify data classification.
- Identify idempotency requirements.
- Reject malformed/incomplete envelopes before execution.

5. Intent & Task Understanding

- Parse structured task intent.
- Determine required capability.
- Map task type to capability registry.
- Separate primary task from incidental text.
- Detect multi-step execution.
- Detect approval prerequisite.
- Classify informational/generative/analytical/tool/side-effecting work.
- Detect routing-changing ambiguity.
- Never invent missing business facts.
- Escalate when safe routing is impossible.

6. Agent Capability Registry

- Know registered agent IDs and versions.
- Know supported task types.
- Know input schemas.
- Know output schemas.
- Know required tools.
- Know permissions.
- Know accessible data classes.
- Know availability/health.
- Know constraints and restrictions.
- Select only an eligible agent.
- Prevent Phase 1 task from accidentally routing to excluded future-phase agents.

7. Phase 1 Routing Map

Task	Route
Lead qualification / sales conversation	Sales Agent
Call/meeting scheduling	Scheduler Agent
Requirement → quotation construction	Quotation Master Agent
Workflow handoff/dependency tracking	Coordination Agent
Model/provider selection	Orchestrator
Business approval/exception	Policy Engine/Admin — never self-authorize

8. Model & Provider Selection

- Read model/provider registry.
- Evaluate capability fit.
- Evaluate quality requirement.
- Evaluate latency requirement.
- Evaluate cost constraints.
- Evaluate context-window requirement.
- Evaluate tool/function support.
- Evaluate structured-output support.
- Evaluate reliability/availability.
- Evaluate data/privacy constraints.
- Apply configured routing policy.
- Record selection reason and policy version.
- Keep provider credentials isolated.
- Do not choose cheapest route when policy requires higher quality.

9. Routing Decision Algorithm — Micro Steps

- Validate task schema.
- Validate tenant and scope.
- Resolve requested capability.
- Load eligible agent candidates.
- Filter by task support.
- Filter by permissions.
- Filter by data classification.
- Filter by tool availability.
- Filter by output-schema compatibility.
- Filter by agent health/availability.
- Load eligible model/provider candidates.
- Filter models/providers by policy.
- Score/choose according to quality, reliability, latency and cost configuration.
- Select primary route.
- Select fallback route when permitted.
- Persist routing decision.
- Dispatch only after all gates pass.

10. Execution Envelope

- Task ID.
- Correlation ID.
- Tenant/client/lead/project scope.
- Source agent.
- Destination specialist.
- Task type.
- Validated intent.
- Authorized context references.
- Requirement/scope/quote/artifact version references.
- Required output schema.
- Acceptance criteria.
- Tool permissions.

- Policy references.
- Deadline/priority.
- Retry budget.
- Idempotency key.
- Security/data-classification metadata.
- Routing decision metadata.

11. Context & Memory

- Retrieve only relevant authorized context.
- Prefer authoritative structured records over guesses.
- Preserve provenance.
- Pass minimum necessary context.
- Respect tenant/client/project isolation.
- Include prior failure context for retry.
- Include exact version IDs.
- Reject stale/conflicting context where it can affect execution.
- Prevent irrelevant memory from influencing routing.
- Never leak private cross-client context.

12. Tool & Permission Gate

- Check agent permission.
- Check task permission.
- Check tenant integration permission.
- Check user/agent authorization.
- Check data classification.
- Block unauthorized tool calls.
- Treat external responses as untrusted.
- Do not let provider output grant permission.
- Record critical permission decisions.
- Return structured denial/error.

13. Execution Control

- Dispatch task.
- Record execution start.
- Apply timeout.
- Track execution status.
- Track important tool calls.
- Receive structured result.
- Validate task/result correlation.
- Validate schema.
- Validate required fields.
- Validate artifact/version references.
- Validate completion criteria.
- Reject incomplete result.
- Return valid result plus execution metadata.

14. Retry

- Classify failure.
- Determine retry safety.

- Check retry budget.
- Apply backoff.
- Preserve correlation ID.
- Increment attempt.
- Prevent duplicate non-idempotent side effects.
- Carry prior failure context.
- Change provider/route only when allowed.
- Stop after retry exhaustion.
- Emit failure/escalation.

15. Fallback

- Find eligible fallback.
- Confirm equivalent capability.
- Confirm permissions/data policy.
- Confirm required schema support.
- Check cost/quality constraints.
- Preserve business ownership.
- Record primary failure.
- Record fallback decision.
- Execute safely.
- Return trace of attempted routes.
- Escalate if no safe fallback exists.

16. Result & Quality Validation

- Validate output schema.
- Validate required fields.
- Validate correlation/task IDs.
- Validate artifact references.
- Validate version references.
- Validate policy/approval references.
- Check configured quality/confidence metadata.
- Reject unsupported claims where evidence is required.
- Separate model suggestions from authoritative state.
- Prevent generated text from granting permission.
- Return structured result and execution trace.

17. Side-Effect Safety

- Detect side-effecting operation.
- Require idempotency key.
- Confirm authorization.
- Record external request/reference.
- Validate external response.
- Do not blindly retry uncertain side effects.
- Use durable event/outbox where required.
- Reconcile uncertain execution before retry.
- Escalate ambiguous financial/destructive outcomes.

18. Phase 1 Examples

Sales: Coordination requests lead qualification. Orchestrator validates the task, selects Sales, selects an eligible model/provider, dispatches, validates structured output and returns it. It does not become the salesperson.

Scheduler: Coordination requests meeting scheduling. Orchestrator routes to Scheduler after capability, permission and tool checks. Scheduler performs actual availability/booking work. Orchestrator validates the result and returns it.

Quotation: Coordination requests quote generation with exact requirement/scope version. Orchestrator routes to Quotation Master, validates output and preserves execution trace. It does not invent pricing or approve discounts.

19. Human/Admin Escalation

- No eligible agent.
- Ambiguous high-risk routing.
- Permission conflict.
- Policy conflict.
- Repeated execution failure.
- Critical timeout.
- Conflicting authoritative results.
- Unsafe side-effect uncertainty.
- Escalation contains task, context reference, attempted routes, failures, policy references and next-action recommendation.
- Never silently bypass approval.

20. Failure Matrix

Failure	Required behavior
Malformed task	Reject before dispatch.
No capable agent	Escalate; never guess.
Agent unavailable	Permitted fallback or escalation.
Model failure	Retry/fallback according to policy.
Timeout	Safe retry; otherwise reconcile/escalate.
Permission denied	Stop and return authorization error.
Invalid output	Reject; retry/fallback if safe.
Conflicting result	Escalate to authoritative owner.
Provider says success, artifact missing	Treat as incomplete; reconcile.
Uncertain non-idempotent side effect	Do not blindly retry.
Policy conflict	Stop; route to Policy/Admin.

21. Security & Data Governance

- Tenant isolation.
- Least privilege.
- Data-classification-aware routing.
- Authorized memory access.
- Tool permission checks.
- Secret isolation.
- External input treated as untrusted.
- Strong tool/action boundaries.
- No cross-client context leakage.
- Audit routing and permission decisions.
- Protect provider credentials.
- Respect retention/audit requirements.

22. Observability & Audit

- Task ID.
- Correlation ID.
- Routing decision.
- Selected agent.

- Selected model/provider where available.
- Routing reason.
- Policy/configuration version.
- Execution timestamps.
- Attempt count.
- Failure/fallback history.
- Tool authorization decisions where required.
- Result validation.
- Escalation.
- Keep secrets out of logs.
- Make lead-to-execution trace reconstructable.

23. Admin Controls

- Agent capability registry.
- Agent availability/status.
- Model/provider registry.
- Routing priorities.
- Quality/latency/cost policy.
- Fallback rules.
- Retry budgets.
- Timeouts.
- Enable/disable routes.
- View routing traces.
- View execution traces.
- Inspect failures/escalations.
- Pause unsafe route.
- Audit configuration changes.

24. Data Model

AgentCapability: agent_id, version, task_types, input_schema, output_schema, tools, permissions, data_classes, availability, constraints.

RoutingDecision: decision_id, task_id, correlation_id, candidate_agents, selected_agent, candidate_models/providers, selected_model/provider, routing_policy_version, selection_reason, constraints, timestamp.

ExecutionRecord: execution_id, task_id, agent, provider/model, attempt, start/end, status, tool calls, result reference, error, fallback_from/to, validation result.

25. Build Checklist — Recommended Order

A. Registry

- Agent registry
- capability registry
- schemas
- model/provider registry
- tool registry
- permissions metadata

B. Routing

- Task parser
- capability matcher
- eligibility filters
- policy filters

- route selection
- fallback selection
- decision persistence

C. Execution

- Execution envelope
- dispatch
- timeout
- result ingestion
- schema validation
- completion contract

D. Reliability

- Failure classifier
- retry budget
- backoff
- idempotency
- fallback
- dead-letter/escalation
- uncertain-side-effect reconciliation

E. Security

- Tenant isolation
- permission gate
- data classification
- secret isolation
- untrusted input handling
- audit

F. Admin/Observability

- Trace IDs
- routing logs
- execution dashboard
- failure queue
- configuration controls
- metrics

26. Test Checklist

- Sales task routes to Sales.
- Scheduling task routes to Scheduler.
- Quotation task routes to Quotation Master.
- Coordination workflow task is not incorrectly treated as domain work.
- Unavailable agent is filtered.
- Unauthorized agent is filtered.
- Missing required tool causes safe failure/fallback.
- Model selection follows configured policy.
- Output schema is enforced.
- Malformed result cannot advance workflow.
- Retry cannot duplicate side effects.
- Fallback preserves correlation and audit.

- External provider response cannot grant permissions.
- Cross-tenant context is blocked.
- Stale version is detected.
- Timeout is handled safely.
- Repeated failure escalates.
- No-route condition escalates.
- Policy conflict stops execution.
- Every route and execution is traceable.

27. End-to-End Phase 1 Test

Lead event → Coordination task → Orchestrator routing → Sales → meeting request → Coordination task → Orchestrator routing → Scheduler → result → Sales → confirmed requirements → quote task → Orchestrator routing → Quotation Master → approval workflow → negotiation/version revision → exact acceptance → close gate.

- Every task has correlation ID.
- Correct specialist is selected.
- Model/provider policy is applied.
- Context is authorized and versioned.
- Results are schema-valid.
- Failures/retries are traceable.
- No duplicate side effects occur.
- Approval is never bypassed.
- Acceptance is never inferred.
- Final routing/execution audit is complete.

28. Definition of Done

- Capability registry works.
- Agent matching works.
- Phase 1 routing is deterministic under same policy/input.
- Model/provider selection follows policy.
- Execution envelope is validated.
- Permissions are enforced.
- Context is minimum and authorized.
- Structured outputs are validated.
- Retry/fallback are bounded and safe.
- Idempotency protects side effects.
- Failures/escalations are durable.
- Admin can inspect traces.
- Audit chain is complete.
- Sales/Scheduler/Quotation/Coordination integrations pass.
- End-to-end Phase 1 test passes.
- Orchestrator does not duplicate specialist authority.

29. Final Operating Contract

Always: understand task → match capability → filter by policy/permission → select agent → select model/provider → build execution envelope → execute → validate → retry/fallback safely → trace → escalate uncertainty.

Never: invent business facts; choose pricing; approve discounts; act as Sales/Scheduler/Quotation Master; replace Coordination; bypass Admin/Policy; expose secrets; leak context; blindly retry unsafe side effects; allow generated text to directly change authoritative state.